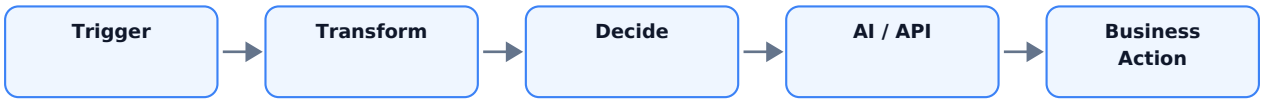
DAY 24

Introduction to n8n

Visual Workflow Automation for AI, APIs, Webhooks, and Business Systems



Course: GPT + AI Agents + RAG + n8n + API Integration + Business Automation

Lecture: 60 minutes | Level: Beginner to Intermediate | Week 4: Automation Engineering

Day 24 moves the course from API and webhook concepts into a visual orchestration platform. Students will learn how n8n represents data, how nodes execute, how workflows branch and loop, how credentials and errors are managed, and where GPT/AI Agents fit inside reliable business automation.

Table of Contents

1. Day 24 Goals and 60-Minute Lecture Plan

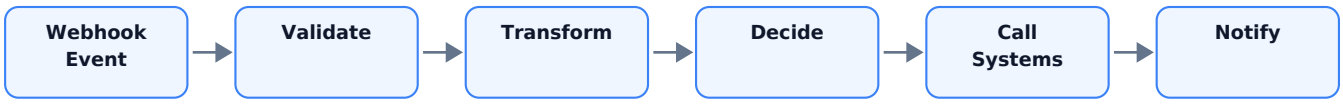
Day 24 answers a practical question: how do we connect triggers, data, AI, APIs, and business actions into one visible, testable workflow? n8n is an orchestration layer. It does not replace APIs, databases, AI models, or business systems; it coordinates them.

Time	Topic	Outcome
0-5 min	Day 23 review	Connect webhook concepts to workflow orchestration
5-12 min	What is n8n?	Understand nodes, connections, triggers, executions
12-20 min	Data model and expressions	Read, map, and transform workflow data
20-30 min	Flow logic	Branch, merge, loop, and control execution
30-38 min	APIs and credentials	Connect external systems safely
38-45 min	Testing, debugging, errors	Build observable, recoverable workflows
45-52 min	AI inside n8n	Use OpenAI/AI Agent in controlled workflow steps
52-57 min	Mini project	Build an AI support-ticket triage workflow
57-60 min	Quiz + homework	Confirm concepts and extend project

Learning outcomes: By the end of the lesson, students should be able to explain workflow anatomy, create a trigger-to-action workflow, read n8n items, use expressions, branch with IF/Switch logic, call an API, understand credentials, inspect executions, design error handling, and identify safe places for AI decisions.

2. From Day 23 Webhooks to Day 24 Orchestration

Day 23 focused on receiving real-time events. A webhook can start work, but a production process usually needs multiple steps after the event arrives.



Without orchestration, developers often write custom glue code for every integration. n8n provides a visual canvas where each step is represented as a node and connections define the flow of data.

2.1 Webhook Is a Trigger, Not the Whole Automation

```
POST /webhook/new-lead
{
  "name": "Taylor",
  "email": "taylor@example.com",
  "message": "We need an AI support system for 20 agents."
}
```

The useful business workflow might still need to validate the payload, normalize fields, enrich company information, classify the lead, write to a CRM, alert sales, and log the execution.

Golden rule: A webhook tells your automation that something happened. The workflow decides what to do about it.

3. What Is n8n?

n8n is a workflow automation platform used to connect applications, APIs, databases, AI models, and custom logic. Workflows are assembled from nodes connected on a visual canvas. Some nodes trigger the workflow; others transform data, make decisions, call services, or perform actions.

3.1 Core Mental Model

```
WORKFLOW
=
Trigger
+
Data
+
Logic
+
Integrations
+
Actions
+
Error Handling
```

3.2 n8n Is an Orchestrator

Component	Responsibility
n8n	Coordinate workflow steps and data movement
GPT / LLM	Interpret language and generate or classify content
AI Agent	Choose tools/steps where flexible decisions are needed
API	Expose application capabilities and data
Database / CRM	Store authoritative business records
RAG	Retrieve company knowledge
Human	Approve sensitive or exceptional actions

This separation matters: a good system does not ask the LLM to be the database, the scheduler, the authentication system, and the workflow engine at the same time.

4. Workflow Anatomy: Nodes, Connections, and Executions

4.1 Node

A node is one step in the automation. Typical node roles include trigger, transform, decision, API call, database operation, messaging action, AI model call, and error handling.

Node Role	Example Purpose
Trigger	Start workflow from webhook, schedule, chat, or app event
Transform	Rename fields, calculate values, prepare JSON
Decision	IF or Switch based on conditions
Integration	Call Slack, Gmail, CRM, database, or HTTP API
AI	Generate, classify, extract, summarize, or use tools
Flow control	Merge paths, loop through batches, call sub-workflows
Error handling	Stop intentionally or route failures to an error workflow

4.2 Connection

A connection indicates where the output of one node goes next. Connections define the execution path and carry items to downstream nodes.

4.3 Execution

An execution is one run of a workflow. During development, manual executions are useful for testing. Production executions are started by published workflow triggers such as webhooks or schedules.

Current n8n documentation distinguishes manual test executions from production executions, which is important when debugging trigger behavior and webhook URLs.

5. Trigger Nodes: How Workflows Start

Every production automation needs a start condition. A trigger answers: what event should cause this workflow to execute?

Trigger Type	Example	Best For
Manual Trigger	Click Execute workflow	Development and testing
Schedule Trigger	Every weekday at 08:00	Reports, sync jobs, recurring checks
Webhook	Incoming HTTP request	Real-time events
App Trigger	New row / new message / CRM event	SaaS event automation
Chat Trigger	Incoming chat interaction	Conversational AI workflows

5.1 Choose Event-Driven When Possible

If a source system can notify you in real time, a webhook or native trigger is often more efficient than polling every minute. If no event exists, scheduling may be appropriate.

> Design question: Is the workflow triggered by time, an event, a user request, or another workflow? Answer this before adding AI.

6. n8n Data Model: Items and JSON

A beginner must understand n8n data before building large workflows. n8n passes data between nodes as an array of items. Each item commonly contains a json object; file-like data may be handled as binary data.

```
[
  {
    "json": {
      "name": "Taylor",
      "email": "taylor@example.com",
      "priority": "high"
    }
  },
  {
    "json": {
      "name": "Jordan",
      "email": "jordan@example.com",
      "priority": "medium"
    }
  }
]
```

6.1 Why Items Matter

- Many nodes process each incoming item independently.
- A node may output one item, multiple items, or transformed items.
- Expressions often read properties from the current item.
- Loops and batching matter when processing many items or rate-limited APIs.

Current n8n documentation explicitly describes data passed between nodes as arrays of objects/items, which is why inspecting the JSON output panel is a core debugging skill.

7. Expressions: Dynamic Values Between Nodes

Expressions let a node use data from the current execution rather than hard-coded text. Expressions are one of the most important n8n skills because they connect outputs from one node to inputs of another.

7.1 Basic Examples

```
{{$json.email}}
{{$json.customer.name}}
{{$json.amount}}
{{$json.firstName + " " + $json.lastName}}
```

7.2 Previous Node Reference

```
{{ $('Webhook').item.json.body.email }}
{{ $('Normalize Lead').item.json.company }}
```

Exact expression syntax can depend on the data shape and item linkage. The editor expression picker is valuable because it exposes available input and previous-node data.

7.3 Expression vs Code Node

Need	Use
Insert one field into another node	Expression
Simple date/string arithmetic	Expression
Complex mapping over many fields	Code or dedicated data node
Custom algorithm / multi-step transformation	Code node
Simple field rename / set values	Edit Fields / mapping

Rule: Prefer the simplest transformation mechanism that remains readable. Do not write code for every one-line mapping.

8. Data Mapping and Normalization

Integrations rarely use identical field names. One system might send customerEmail, another expects email, and a CRM may require primary_email. Normalization creates a stable internal schema.

```
Incoming Webhook
{
  "full_name": "Taylor Kim",
  "mail": "taylor@example.com",
  "msg": "Need enterprise support automation"
}

Normalized Item
{
  "name": "Taylor Kim",
  "email": "taylor@example.com",
  "message": "Need enterprise support automation"
}
```

8.1 Why Normalize Early?

- Downstream nodes depend on one predictable schema.
- Prompt templates become simpler.
- API payloads become easier to build.

- Changing the source system affects fewer nodes.
- Logs and tests become easier to compare.

A professional workflow often has a normalization boundary close to the trigger. This reduces coupling between external payloads and internal business logic.

9. Flow Logic: IF, Switch, Merge, and Branching

9.1 IF Node

Use IF for a binary decision: condition true or false.

```
IF lead_score >= 70
   TRUE  -> High-priority sales route
   FALSE -> Standard nurture route
```

9.2 Switch Node

Use Switch when one value can map to multiple routes.

```
category
  billing   -> Billing queue
  shipping  -> Shipping queue
  sales     -> Sales queue
  other     -> General queue
```

9.3 Merge

Use merge/join behavior when two paths must be combined or synchronized before later processing. The exact mode should match the data relationship: append, combine, or wait for branches as appropriate.

Design principle: Put objective routing rules in flow logic when possible. Use AI to classify ambiguous text; use IF/Switch to enforce deterministic routing.

10. Looping and Batch Processing

Many workflows process lists: contacts, invoices, tickets, products, or rows. n8n can process each item automatically, and the Loop Over Items node can divide incoming items into batches when explicit batch control is needed.

10.1 Why Batch?

- Respect external API rate limits.
- Control memory and workload size.
- Process thousands of records in smaller groups.
- Insert waits between requests when required.
- Make recovery easier when one batch fails.

```
1000 CRM contacts
      |
Loop Over Items (batch size 50)
      |
Enrich -> Score -> Update CRM
      |
Next batch
      |
Done
```

A common beginner mistake is assuming every list needs a manual loop. First understand how the chosen node processes multiple incoming items; add Loop Over Items when you need batching or explicit loop control.

11. HTTP Request Node: Connect Almost Any API

When n8n does not have a dedicated integration node, the HTTP Request node is a universal bridge to REST APIs.

Field	Example
Method	GET / POST / PATCH / DELETE
URL	https://api.example.com/v1/customers
Authentication	Credential or header-based auth
Headers	Content-Type, Authorization, custom headers
Query Parameters	status=active&page;=2
Body	JSON payload for create/update requests
Response	JSON, text, file, or configured response format

11.1 Example Business API Call

```
POST https://api.example.com/v1/tickets
Authorization: Bearer <credential-managed-token>
Content-Type: application/json

{
  "customer_email": "{{$json.email}}",
  "category": "{{$json.category}}",
  "priority": "{{$json.priority}}",
  "summary": "{{$json.summary}}"
}
```

Use credentials rather than hard-coding secrets directly in fields or prompts. Keep authentication separate from model-visible text.

12. Credentials and Secret Management

Credentials are the authentication material used by integration nodes: API keys, OAuth tokens, basic auth, and other secrets. The workflow should reference a credential configuration instead of embedding raw secrets into expressions or AI prompts.

12.1 Security Rules

- Never place API keys in prompts or customer-visible output.
- Use least-privilege API credentials.
- Separate development and production credentials.
- Rotate secrets when compromised or when policy requires it.
- Do not log sensitive values unnecessarily.
- Restrict who can edit workflows that have powerful credentials.

Critical: AI instructions are not an authorization system. Even if GPT says an action is allowed, the underlying credential and workflow must enforce actual permissions.

13. Testing Workflows: Manual Runs, Pinned Data, and Mocking

Reliable workflow development is iterative. Test one path at a time rather than building 25 nodes and running everything only at the end.

13.1 Recommended Build Cycle

1. Add a trigger or sample input.
2. Run the first node and inspect its output JSON.
3. Pin or preserve representative test data when useful.
4. Add one transformation or action.
5. Execute the new node and inspect output.
6. Add branching paths and test each branch.
7. Test failure cases, not just happy paths.
8. Publish only after production triggers, credentials, and error handling are verified.

13.2 Pinned / Mock Data

Pinned data is useful for development because you can reuse known input while configuring downstream nodes. This reduces repeated calls to live services and makes testing reproducible.

> Do not confuse pinned test data with live production truth. Before production, test with realistic data and confirm that the published workflow uses real inputs.

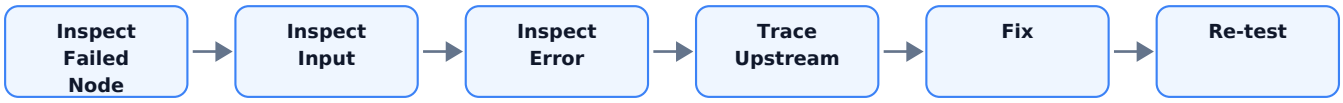
14. Executions and Debugging

When a workflow fails, do not immediately change the prompt. First identify which node failed, what input it received, what it attempted, and what output or error it produced.

14.1 Debugging Questions

- Did the trigger run?
- Did the expected item reach this node?
- Is the expression reading the correct field?
- Did authentication succeed?
- Did the API return 2xx, 4xx, or 5xx?
- Did an AI node return the expected structured shape?
- Did a branch condition route correctly?
- Was data lost or renamed in a previous node?
- Did the workflow retry or continue on error unexpectedly?

14.2 Debugging Sequence



The execution view is a business automation equivalent of a debugger. Learn to read data at every node boundary.

15. Error Handling and Error Workflows

Production workflows must be designed for failures. APIs time out, credentials expire, payloads change, records disappear, and AI output may be invalid.

Failure	Likely Handling
Temporary HTTP 5xx	Limited retry / backoff
HTTP 401/403	Stop; fix credentials or permission
Record not found	Ask user, branch, or create exception case
Invalid payload	Validate and reject/quarantine
AI invalid output	Retry with validation boundary or route to review
Sensitive action	Human approval before write
Unexpected workflow error	Error workflow + alert + log

15.1 Error Trigger

n8n supports error workflows using an Error Trigger. Current documentation notes that the Error Trigger is for automatic workflow errors, while development/manual test behavior can differ. You can also deliberately stop a workflow with a custom error when a business invariant is violated.

```
Main Workflow
   |
   +-- success --> Continue
   |
   +-- error ----> Error Workflow
                     |
                     +--> Log execution
                     +--> Alert owner
                     +--> Create incident/ticket
```

16. Designing Idempotent Business Workflows

Webhooks and retries may deliver the same event more than once. A workflow that creates invoices, refunds, tickets, or emails must consider duplicate execution.

16.1 Idempotency Pattern

```
Receive event with event_id
        |
Check event log/table
   |              |
Exists?         New?
   |              |
Stop          Mark processing
                  |
              Perform action
                  |
              Mark complete
```

16.2 Good Idempotency Keys

- Webhook event ID from source system.
- Order ID + action type + version.
- Invoice ID from accounting system.
- A client-provided idempotency key for API actions.

Do not rely on "the model probably will not repeat the call." Duplicate protection belongs in deterministic workflow/application logic.

17. Sub-Workflows and Reusable Automation

Large workflows become difficult to test and maintain. n8n supports breaking logic into smaller workflows and executing a sub-workflow from a parent workflow.

17.1 Good Candidates for Sub-Workflows

- Send standardized Slack/Teams alert.
- Normalize customer identity.
- Create CRM activity record.
- Fetch and standardize order data.
- Run a common approval process.
- Reusable AI classification service.

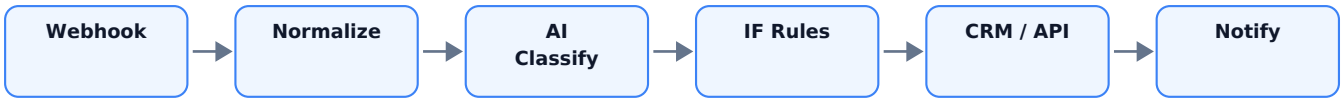
A sub-workflow should have a clear input contract and output contract. Treat it like an internal API.

18. AI in n8n: Where GPT Fits

n8n currently provides AI integration capabilities including OpenAI-related nodes and an AI Agent node. The important architectural question is not "Can AI be inserted here?" but "Does this step require semantic interpretation or generation?"

Task	AI?	Reason
Extract intent from free-form email	Yes	Semantic language understanding
Check amount > 500	No	Deterministic rule
Summarize a long support ticket	Yes	Language transformation
Look up current inventory	No - use API	Live business fact
Draft a personalized response	Yes	Generation
Verify user is authorized	No	Security/auth system

18.1 AI as One Node in a Larger System



Golden rule: Put AI between clear input and validation boundaries. Do not make the entire workflow an opaque prompt.

19. OpenAI Node vs AI Agent Node

For beginners, distinguish a direct model operation from an agentic workflow. A direct OpenAI/model node is appropriate when you know exactly what AI task should happen. An AI Agent node is useful when the model needs to decide which connected tools to use.

Pattern	Use When	Example
Direct model call	One defined AI task	Classify email into category
AI Agent	Model must select tools/steps	Support agent chooses order lookup or policy search
Deterministic workflow	Exact business process	If score >= 70 then assign sales rep

19.1 Current AI Agent Concept in n8n

Current n8n documentation describes the AI Agent node as connecting a chat model with one or more tools, allowing the agent to decide which tools to call. This mirrors the agent concepts from Days 8-13: model + instructions + tools + execution loop.

```
Chat Trigger / Workflow Input
        |
     AI Agent
   /     |      \
Chat   Tool A   Tool B
Model
        |
     Final Output
```

Do not start every automation with an agent. Start with deterministic orchestration and add agent autonomy only when flexible tool selection creates real value.

20. Human Approval and Controlled Automation

n8n is particularly useful for placing AI recommendations inside a controlled process. A model can interpret a request, but a deterministic branch or approval stage can gate sensitive actions.

```
Customer Request
      |
AI extracts intent + amount
      |
Business Rule
 amount > $500 ?
   /          \
 Yes          No
  |            |
Human          Continue
Approval
  |
Refund API
```

20.1 Approval Data Should Be Structured

```
{
  "action": "refund",
  "amount": 750,
  "reason": "duplicate_charge",
  "requires_approval": true
}
```

Store the actual approval result in a trusted system or workflow state. Do not rely on the model remembering that "someone approved it" in conversational text.

21. A Professional Workflow Architecture

A mature workflow separates ingestion, interpretation, business rules, side effects, and observability.

```
TRIGGER
  |
VALIDATE INPUT
  |
NORMALIZE DATA
  |
AI INTERPRETATION (if needed)
  |
SCHEMA / OUTPUT VALIDATION
  |
DETERMINISTIC BUSINESS RULES
  |
READ TOOLS / API LOOKUPS
  |
APPROVAL GATE (if needed)
  |
WRITE ACTION
  |
LOG + METRICS + NOTIFICATION
```

21.1 Why This Structure Works

- Each layer has one responsibility.
- AI output cannot directly bypass policy.
- Failures are easier to locate.
- Testing can target each boundary.
- Business rules remain auditable.
- Integrations can be replaced without rewriting every prompt.

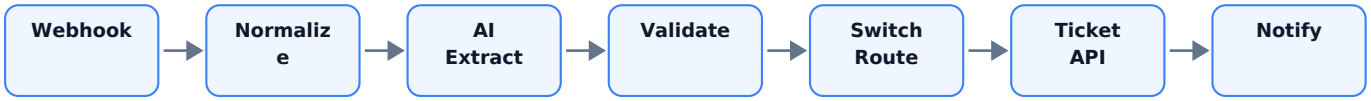
22. Day 24 Mini Project - AI Support Ticket Triage Workflow

The mini project turns an incoming support message into a validated, classified, routed business record. It intentionally combines the concepts from Days 3-5, Day 22 APIs, Day 23 webhooks, and Day 24 n8n.

22.1 Business Scenario

A company receives support requests through a web form. It wants to classify each request, assign a priority, route the case to the correct team, create a ticket through an API, and alert a human when the message is high risk.

22.2 Target Architecture



The AI step performs fuzzy language interpretation. n8n performs orchestration, data mapping, deterministic routing, API calls, and error handling.

23. Mini Project - Input and Output Contracts

23.1 Incoming Webhook Payload

```
{
  "customer_name": "Taylor Kim",
  "customer_email": "taylor@example.com",
  "message": "I was charged twice and need this fixed today."
}
```

23.2 Normalized Internal Schema

```
{
  "name": "Taylor Kim",
  "email": "taylor@example.com",
  "message": "I was charged twice and need this fixed today.",
  "received_at": "<workflow timestamp>"
}
```

23.3 AI Structured Output

```
{
  "summary": "Customer reports duplicate charge.",
  "category": "billing",
  "priority": "high",
  "sentiment": "negative",
  "requires_human_review": true,
  "recommended_action": "billing_review"
}
```

Treat this JSON as an interface between AI interpretation and workflow logic. Validate category values and required fields before routing.

24. Mini Project - Node-by-Node Build

#	Node	Purpose
1	Webhook	Receive support form POST
2	Edit Fields / Normalize	Map source fields to stable internal names
3	OpenAI / AI step	Extract category, priority, summary, review flag
4	Validation IF	Reject or review invalid AI output
5	Switch	Route billing/shipping/sales/general
6	HTTP Request	Create ticket in external support API
7	IF high risk	Check deterministic escalation rule
8	Notification	Alert support lead when necessary
9	Respond to Webhook	Return accepted/ticket result if synchronous response required

24.1 Workflow Expression Examples

```
Email field:
{{$json.email}}

Category field:
{{$json.category}}

Ticket title:
Support - {{$json.category}} - {{$json.name}}
```

25. Mini Project - AI Classification Prompt

Use a narrow prompt with explicit categories and no permission to execute business actions.

```
SYSTEM / INSTRUCTION
You classify customer support requests.

Allowed categories:
- billing
- shipping
- sales
- product_question
- account
- other

Priority:
- low
- medium
- high

Set requires_human_review=true for:
- legal threats
- payment disputes
- security/account takeover concerns
- threats or safety issues
- requests outside policy

Do not invent customer, order, payment, or policy facts.
Return only the required structured fields.

USER MESSAGE:
{{$json.message}}
```

In production, prefer the model/tool/node features that support structured output or explicit schemas when available, then validate the result before using it in deterministic routing.

26. Mini Project - Deterministic Business Rules

The AI suggests a priority and review flag, but the workflow can still enforce non-negotiable business rules.

Condition	Workflow Rule
category = billing AND message contains duplicate charge indicator	Route to billing; human review
requires_human_review = true	Notify human queue
category not in allowed set	Reject AI output / fallback route
customer_email missing	Do not create external ticket; ask for data
Ticket API returns 5xx	Retry according to policy, then error route
Ticket API returns 401/403	Stop and alert operator

Key lesson: AI can interpret the message, while workflow rules decide what the business system is allowed to do.

27. Mini Project - Ticket API Payload

```
POST /tickets

{
  "requester": {
    "name": "{{$json.name}}",
    "email": "{{$json.email}}"
  },
  "subject": "{{$json.summary}}",
  "category": "{{$json.category}}",
  "priority": "{{$json.priority}}",
  "description": "{{$json.message}}"
}
```

27.1 Keep External Payloads Separate

Do not send the entire internal n8n item to every API. Build the exact payload the destination requires. This limits accidental data leakage and makes integration contracts easier to test.

27.2 Tool/API Error Result

```
{
  "success": false,
  "error": {
    "code": "TICKET_API_UNAVAILABLE",
    "message": "Ticket system returned 503"
  }
}
```

Downstream nodes should branch on structured success/error state instead of hoping the next step can infer what happened.

28. Mini Project - Test Matrix

Test	Expected Category	Expected Flow
"Where is order A123?"	shipping	Normal shipping route
"I was charged twice."	billing	High priority + human review
"Can your product integrate with Salesforce?"	sales/product	Sales or product route per policy
"Thank you, issue solved."	other	Low priority / no escalation
Missing email	any	Validation branch before ticket API
Prompt injection inside message	based on actual business content	Ignore instruction to override system rules
Ticket API 503	any	Retry/error handling path
Unknown AI category	invalid	Fallback/review

28.1 What to Record

- Input payload.
- AI output.
- Branch selected.
- API request payload.
- API response.
- Execution status.
- Whether human review was correctly triggered.

29. Common n8n Beginner Mistakes

Mistake	Why It Hurts	Fix
Hard-code values everywhere	Workflow breaks when input changes	Use expressions and normalized schema
Build huge workflow at once	Difficult to debug	Add/test node by node
Put API keys in text fields	Security risk	Use credentials
Use AI for exact rules	Inconsistent business behavior	Use IF/Switch/code
Ignore item data shape	Expressions return undefined/null	Inspect node JSON
No error path	Silent data loss / stuck automations	Design retries and error workflow
No duplicate protection	Repeated side effects	Use event IDs/idempotency
One giant workflow	Maintenance pain	Use reusable sub-workflows
Publish without failure tests	Production incidents	Test negative paths

30. Five Business Automation Patterns to Memorize

Pattern 1 - Trigger -> Transform -> Action

```
New CRM Lead -> Normalize -> Send Slack Alert
```

Pattern 2 - Trigger -> AI -> Rule -> Action

```
New Email -> AI Classify -> IF Priority -> Create Ticket
```

Pattern 3 - Trigger -> API Enrichment -> AI -> CRM

```
Lead Webhook -> Company API -> AI Summary -> CRM Update
```

Pattern 4 - Scheduled Batch Automation

```
Schedule -> Fetch Records -> Loop Batches -> Transform -> Update
```

Pattern 5 - Agent + Controlled Tools

```
Chat Trigger -> AI Agent -> Read Tools -> Approval Gate -> Write Action
```

Most paid automation projects are combinations of a small number of reusable patterns. Learn the patterns, then map each client process onto them.

31. Client Discovery Questions for n8n Projects

1. What event starts the process?
2. Which systems provide input data?
3. Which system is the source of truth for each field?
4. What decisions are deterministic?
5. Which decisions require language understanding or AI?
6. Which actions create side effects?
7. Which actions require human approval?
8. What happens if an API is unavailable?
9. Can the same event arrive twice?
10. How should failures be reported?
11. What volume of items is expected per day?
12. Are there API rate limits?
13. What data is sensitive?
14. Who should have access to credentials?
15. What metrics define success?

A strong automation architect starts with the process and source-of-truth map, not with a favorite node or AI model.

32. Reliability, Cost, and Operations Metrics

Metric	What It Tells You
Workflow success rate	How often executions complete correctly
Error rate by node	Where integrations are failing
Average execution duration	Latency and bottlenecks
AI calls per execution	AI cost/complexity
External API calls	Integration load and rate-limit pressure
Retries per execution	Instability or poor error classification
Manual-review rate	How much human intervention remains
Duplicate prevented count	Effectiveness of idempotency controls

For business automation, "it worked in my demo" is not an operational metric. Measure reliability across repeated executions and failure cases.

33. Day 24 Quiz - 15 Questions

1. What is n8n primarily used for?

Answer: Workflow orchestration and automation across systems.

2. What starts a workflow?

Answer: A trigger such as webhook, schedule, app event, chat, or manual trigger.

3. What is an n8n item?

Answer: A unit of data passed between nodes, commonly containing a json object.

4. Why use expressions?

Answer: To map dynamic execution data into node parameters.

5. IF vs Switch?

Answer: IF is typically binary; Switch is useful for multiple routes.

6. Why normalize input early?

Answer: To create a stable internal schema for downstream nodes.

7. When is Loop Over Items useful?

Answer: Batching/explicit loops, especially for large lists or rate-limited APIs.

8. What does HTTP Request provide?

Answer: A generic way to connect APIs without a dedicated node.

9. Where should API keys live?

Answer: Credential management, not prompts or hard-coded text.

10. What is an execution?

Answer: One run of a workflow.

11. Why use pinned data?

Answer: Repeatable testing while building downstream nodes.

12. Why design an error workflow?

Answer: To log, alert, or recover when automatic workflows fail.

13. What is idempotency?

Answer: Preventing repeated delivery/retries from causing duplicate side effects.

14. AI or code for amount > $500?

Answer: Deterministic code/flow logic.

15. When should an AI Agent be used?

Answer: When flexible tool/step selection is genuinely needed, not for every automation.

34. Classroom Exercises

Exercise A - Data Mapping

Incoming JSON uses mail, full_name, and msg. Design normalized fields email, name, and message, then write the expressions you would use.

Exercise B - Routing

Design a Switch with four categories: billing, shipping, sales, other. Define the destination node for each route.

Exercise C - Error Policy

For HTTP 429, 401, 404, and 503, decide whether to retry, stop, ask for data, or escalate.

Exercise D - AI Boundary

Given 10 workflow decisions, classify each as AI, deterministic rule, API lookup, RAG, or human approval.

Exercise E - Architecture

Draw an n8n workflow for: new customer email -> classify -> retrieve order -> search policy if needed -> draft response -> human review for high-risk cases -> send reply.

35. Detailed Homework

Homework 1 - Build a 7-Node Workflow

Create a workflow with Manual Trigger -> sample input -> normalization -> classification placeholder -> IF/Switch -> simulated API action -> final output. The classification can initially be deterministic mock data if no API credentials are available.

Homework 2 - Expression Practice

Create 15 expressions that reference current-item fields, nested fields, concatenated strings, and at least three previous-node fields.

Homework 3 - Error Matrix

Create a table for 10 failure types with retry/stop/escalate behavior and maximum retry count.

Homework 4 - Support Triage Project

Implement the Day 24 mini project using a webhook or Manual Trigger. Add at least six test cases and capture execution screenshots/results for your portfolio.

Homework 5 - Reusable Sub-Workflow

Create one reusable notification sub-workflow that accepts: severity, title, message, source_workflow, execution_id. Return a structured success result.

Homework 6 - Architecture Review

Take one real business process and produce: trigger, input schema, AI steps, deterministic rules, integrations, write actions, approval points, error paths, and success metrics.

36. Interview and Upwork Preparation

Question: What is the difference between n8n and GPT?

GPT handles language/AI reasoning tasks. n8n orchestrates workflow execution, data movement, branching, integrations, and business actions.

Question: How does data move through n8n?

Nodes pass items to downstream nodes. Items usually expose JSON data that can be mapped with expressions.

Question: How would you secure API integrations?

Use managed credentials, least-privilege permissions, separate environments, avoid secret logging, and keep secrets out of prompts and ordinary workflow fields.

Question: When would you use an AI Agent instead of a normal workflow?

When the correct next tool or action depends on flexible interpretation and cannot be cleanly expressed as a small deterministic decision tree. Even then, sensitive side effects should remain policy-controlled.

Question: How do you debug an n8n workflow?

Inspect the failed execution, identify the failed node, verify its input and expressions, inspect the external error response, trace upstream data shape, patch one cause, and re-test affected paths.

Question: What makes an automation production-ready?

Validation, credential security, deterministic business rules, idempotency, retries, error handling, observability, realistic tests, human approval for sensitive actions, and measurable success criteria.

37. Portfolio Project Packaging

Do not present the mini project as "I connected some nodes." Present it as a business system with a measurable objective.

37.1 Suggested Case Study Title

AI-Powered Customer Support Triage Automation with n8n

37.2 Case Study Sections

1. Business problem and baseline.
2. Workflow architecture diagram.
3. Input/output schemas.
4. AI classification design.
5. Deterministic routing and approval rules.
6. External integrations.
7. Error and duplicate handling.
8. Test dataset and results.
9. Security assumptions.
10. Business metrics and future improvements.

37.3 Strong Portfolio Metrics

- Classification accuracy on a labeled test set.
- Percentage of tickets auto-routed correctly.
- Average execution time.
- Number of manual steps removed.
- Error recovery success rate.
- Human-review rate for high-risk cases.

38. Day 24 Learning Checklist

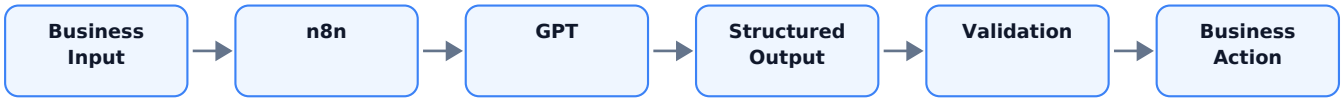
- [ ] Explain n8n as an orchestration platform.
- [ ] Identify trigger, action, logic, and integration nodes.
- [ ] Explain n8n items and JSON data.
- [ ] Use expressions to map dynamic values.
- [ ] Normalize incoming data.
- [ ] Design IF and Switch routes.
- [ ] Explain batch processing and Loop Over Items.
- [ ] Use the HTTP Request node conceptually.
- [ ] Explain why credentials should manage secrets.
- [ ] Distinguish manual and production executions.

- [ ] Use pinned/mock data for development.
- [ ] Debug by inspecting node inputs/outputs.
- [ ] Design an error workflow.
- [ ] Explain idempotency.
- [ ] Break large automations into sub-workflows.
- [ ] Place AI only where semantic reasoning is needed.
- [ ] Distinguish direct AI calls from an AI Agent.
- [ ] Design human approval gates.
- [ ] Build a support triage workflow architecture.

Day 24 final formula: Reliable n8n Automation = Trigger + Clean Data + Simple Flow Logic + Secure Integrations + AI Where Useful + Validation + Error Handling + Observability.

39. Preview - Day 25: n8n + GPT

Day 25 moves from general n8n workflow skills to focused AI workflow implementation. The central question becomes: How do we make GPT a reliable component inside n8n rather than a free-form text box?



Day 25 Topics

- OpenAI/model credentials and node setup concepts.
- Prompt templates with n8n expressions.
- System instructions vs dynamic user data.
- Structured JSON output for workflows.
- Extraction, classification, summarization, and drafting patterns.
- Validation and fallback paths.
- Retries and model/API failures.
- AI cost and token-control concepts.
- Batching AI calls safely.
- RAG and API context inside an n8n flow.
- Human review before sensitive actions.
- Complete AI email-processing automation.

Day 25 project: AI Email Operations Workflow - receive email -> normalize -> GPT extracts intent/entities -> retrieve needed business data -> deterministic route -> draft response -> human approval when required -> send/update system.

40. References and Further Reading

n8n - Expressions for data transformation
https://docs.n8n.io/build/work-with-data/transform-data/expressions-for-data-transformation

n8n - Understand n8n data structure
https://docs.n8n.io/build/work-with-data/understand-n8ns-data-structure

n8n - Webhook node
https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.webhook

n8n - Loop Over Items / flow logic
https://docs.n8n.io/build/flow-logic/loop

n8n - Handle errors gracefully
https://docs.n8n.io/build/flow-logic/handle-errors-gracefully

n8n - Error Trigger
https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.errortrigger

n8n - Manual Trigger
https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.manualworkflowtrigger

n8n - Execution types
https://docs.n8n.io/build/understand-workflows/understand-executions/types-of-executions

n8n - Save and publish workflows
https://docs.n8n.io/build/understand-workflows/save-and-publish-workflows

n8n - AI Agent node
https://docs.n8n.io/integrations/builtin/cluster-nodes/root-nodes/n8n-nodes-langchain.agent

n8n - OpenAI node
https://docs.n8n.io/integrations/builtin/app-nodes/n8n-nodes-langchain.openai

n8n - Integrate AI
https://docs.n8n.io/build/integrate-ai

n8n - Break workflows into smaller parts
https://docs.n8n.io/build/flow-logic/break-workflows-into-smaller-parts

Documentation evolves. For production implementation, verify node options and platform behavior against the current official documentation at the time you build or deploy.